

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

Operating System (BCSE303P)

Course Project – End-to-End Project Guide

L15+L16

Faculty: Dr. Anisha M. Lal

Adaptive CPU Scheduling Using Predicted Burst Lengths

Prepared by

Shivaansh Aggarwal	24BCE0883
Devansh Sharma	24BCE0717

Contents

1. The project in one page
2. Concepts used in the project
3. Architecture
4. What happens during one simulation run
5. Code walkthrough, module by module
6. The prediction model
7. The adaptive scheduler in detail, with worked examples
8. Experiments and evaluation pipeline
9. Results and what they mean
10. How to run everything
11. Where everything is saved
12. Limitations and future work
13. Questions to be ready for

1. The project in one page

What it is. A CPU scheduler that predicts how long each process will run on its next CPU burst, and uses that prediction twice: to decide **which** process runs next, and to decide **how long** it may run (the time quantum). The time quantum is recalculated continuously while the system runs, instead of being fixed by hand.

Why. SJF gives the lowest average waiting time but needs the next burst length, which nobody knows in advance. The usual estimate, exponential averaging, reacts slowly when a process changes behaviour. Round Robin needs a quantum, and the textbook rule for choosing one (about 80% of bursts should finish within one quantum) is normally applied once by a person and never revisited.

How it was built. Everything is in Python. A discrete-event simulator models one CPU with processes that alternate CPU bursts and I/O. Eleven scheduler configurations run on exactly the same workloads: FCFS, SJF, SRTF, Round Robin (four quanta), MLFQ, an oracle that knows the future, and two versions of the proposed scheduler. A gradient-boosting model, trained on generated process histories, predicts the next burst.

Result in one line. On the mixed workload, the realistic case, the proposed scheduler cuts average waiting time by **27%** compared with SJF using exponential averaging. It is better on all 10 seeds, closes **54%** of the gap to the oracle, lowers the worst-case wait by **23%**, and uses **61%** fewer context switches than Round Robin with a 5 ms quantum. It does not help on workloads made only of short random bursts, and the documents say so openly.

2. Concepts used in the project

Term	Meaning in this project
CPU burst / I/O burst	A period of computing / a period of waiting for I/O. Every process here is *cpu, io, cpu, io, …, cpu*.
Waiting time (W)	Time spent in the ready queue: turnaround − CPU time − I/O time. The main metric.
Turnaround time (T)	Completion time − arrival time.
Response time (R)	First time on the CPU − arrival time. Matters for interactive programs.
Context switch	Moving the CPU to a different process. Costs 0.1 ms in the simulator.
FCFS	Runs in arrival order. Convoy effect: 24, 3, 3 ms bursts give 17 ms average wait.
SJF / SRTF	Shortest burst first; SRTF also preempts when a shorter job arrives. Optimal on average, but needs burst lengths.
Exponential averaging (EMA)	$\tau_{n+1} = \alpha t_n + (1-\alpha)\tau_n$, $\alpha = 0.5$. The classical burst estimate.
Round Robin (RR), quantum	Each process gets at most q ms, then goes to the back of the queue.
MLFQ	Multilevel feedback queue: processes that use their whole quantum drop a level; periodic boost avoids starvation.
Starvation, ageing	A process waiting forever; ageing improves a process's priority the longer it waits.
Oracle	SRTF that knows every true burst length. The best-case reference, impossible in practice.
MAE	Mean absolute error of the burst prediction, in ms.
Seed	A number that fixes the random workload, so a run can be repeated exactly.

3. Architecture

The key design decision is that the **simulation engine never makes a scheduling decision**. It owns the clock, the event queue and the process control blocks (PCBs), and whenever a decision is needed it calls the **policy interface**. Any scheduler that implements the interface can be plugged in. This guarantees that every scheduler is tested on identical arrivals, bursts, I/O times and switch costs, so differences in the results come only from the decisions.

Inside the adaptive scheduler, the **feature extractor** turns a process's history into 12 numbers, the **burst predictor** estimates the next burst, the **classifier** labels the process type, the **ready-queue ordering** applies the ageing formula, and the **quantum controller** recomputes the time slice. When a burst completes, the **feedback loop** records the error and stores the example for refitting. After a run, the **metrics collector** computes all measures and the **analysis** step produces tables and figures.

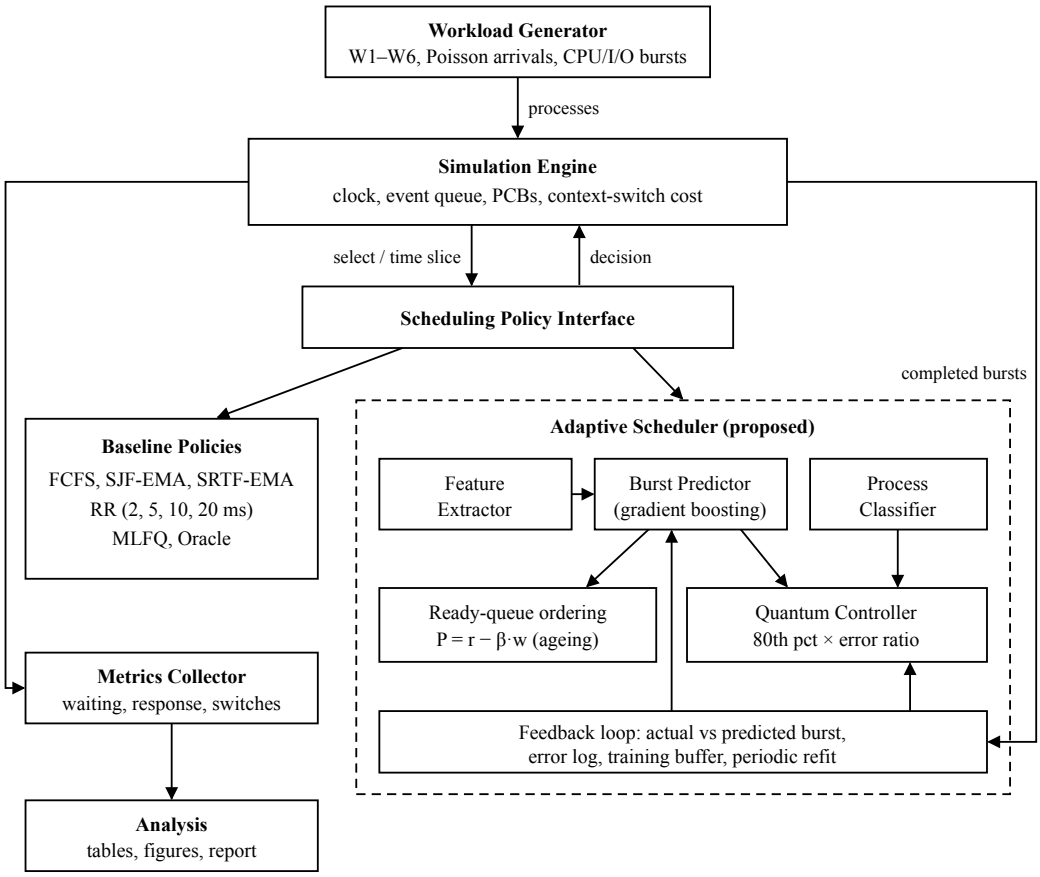


Fig. 1. System architecture

4. What happens during one simulation run

Following one process through the engine (`scheduler/engine.py`) makes the whole system easy to understand:

1. **Start.** The engine puts an `ARRIVAL` event for every process into a priority queue ordered by time.
2. **Arrival.** When an arrival event is popped, the process's first burst is loaded (`_start_burst`) and it joins the ready queue (`_make_ready`). The policy's `on_ready` hook runs; for the adaptive scheduler, this is where the next burst is predicted.
3. **Preemption check.** For preemptive policies (SRTF, MLFQ, oracle), the engine asks `should_preempt` whether the newcomer should interrupt the running process.
4. **Dispatch.** After all events at the current instant are handled, if the CPU is idle the engine calls `select` to pick a process and `time_slice` to get its quantum. If the process differs from the last one, a 0.1 ms switch cost is charged. A `SLICE_END` event is scheduled at *start + min(quantum, remaining burst)*.
5. **Slice end.** If the burst is finished, `on_burst_complete` runs (feedback loop), and an `IO_DONE` event is scheduled after the I/O time. If the quantum ran out instead,

`on_quantum_expired` runs and the process goes back to the ready queue.

6. **I/O done.** The next burst is loaded and the process becomes ready again. Steps 2–6 repeat.
7. **Finish.** After the last burst, the completion time is recorded. When the event queue is empty, the engine returns a `SimResult` with all PCBs, the timeline and the switch count.

Two details keep the engine correct. First, all events at the same time are processed before a decision is made, so processes arriving together are all considered (this is what makes the SJF example give exactly 3 ms). Second, when a slice is cut short by preemption, its pending `SLICE_END` event is cancelled with a token, so it cannot fire later by mistake.

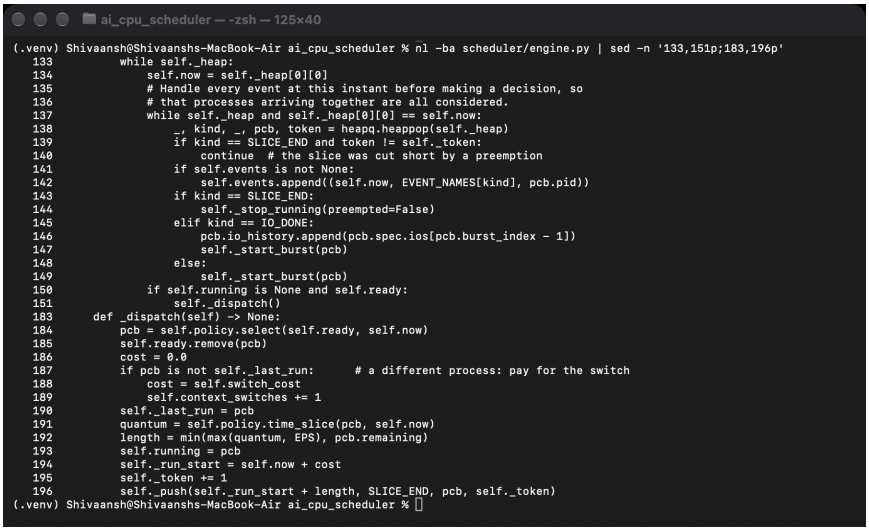


Fig. 2. The event loop and dispatch in scheduler/engine.py (Terminal, with line numbers)

5. Code walkthrough, module by module

File	What is inside
scheduler/workload.py	`ProcessSpec` (one process: arrival, bursts, I/O waits, true class of each burst). `make_process` draws one process of a type. `generate_workload(profile, seed)` builds W1–W6 with Poisson arrivals at 85% load. `shifting_workload` builds the changing workload used for the quantum figure. `simple_processes` builds textbook examples.
scheduler/engine.py	`PCB` (run-time state), `Policy` (the interface: `on_ready, select, time_slice, should_preempt, on_quantum_expired, on_preempt, on_burst_complete, stats`), `Simulator` (event loop), `SimResult`.
scheduler/baselines.py	`FCFS`, `RoundRobin`, `SJF`, `SRTF`, `MLFQ`, `Oracle`; `PredictivePolicy` (shared code for policies that use a predictor); `estimated_remaining`.
scheduler/features.py	`ema_estimate`, `extract_features` (the 12 features), `build_dataset` (one row per burst for training).

scheduler/predictor.py	`BurstPredictor` (base: times each prediction, logs each error), `EMAPredictor`, `MLPredictor` (learned model with online refit), `ProcessClassifier`.
scheduler/quantum.py	`compute_quantum` (the percentile rule) and `QuantumController` (epochs, smoothing, error ratios, class modulation).
scheduler/adaptive.py	`priority_score` and `AdaptiveScheduler` (ordering with ageing, uses the controller for the quantum).
scheduler/metrics.py	`per_process` (T, W, R per process), `summarize` (one result row per run), `jain_index`, `count_starved`, `oracle_gap_closed`.
scheduler/configs.py	The 11 configuration names, default parameters (β = 0.03, p = 80, E = 50, γ = 0.3), ablation variants, `make_policy(name)` and `run_policy(specs, name)`.
train_models.py	Builds the training data, compares 4 regressors and 2 classifiers, saves the chosen models.
tune_beta.py	Chooses the ageing weight β on separate seeds.
run_experiments.py	Runs the full experiment in parallel across CPU cores and saves raw CSVs.
analyse.py	Turns the CSVs into REPORT.md, a summary table and 8 figures.
demo.py	10-second live demo in three parts (`--part 1/2/3`).
tests/	37 pytest tests (engine, policies, predictors, adaptive scheduler, metrics, workloads).

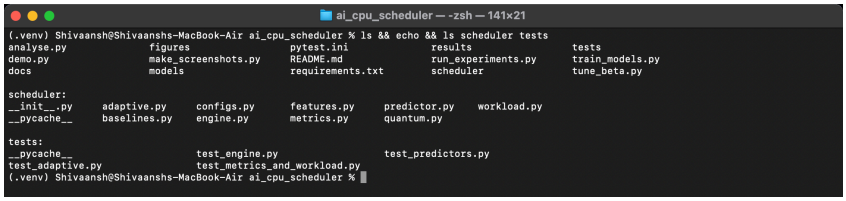


Fig. 3. Project files and the scheduler package

To see how small a policy is, this is the complete Round Robin and SJF code. Round Robin always picks the front of the queue with a fixed quantum; SJF picks the smallest predicted burst:

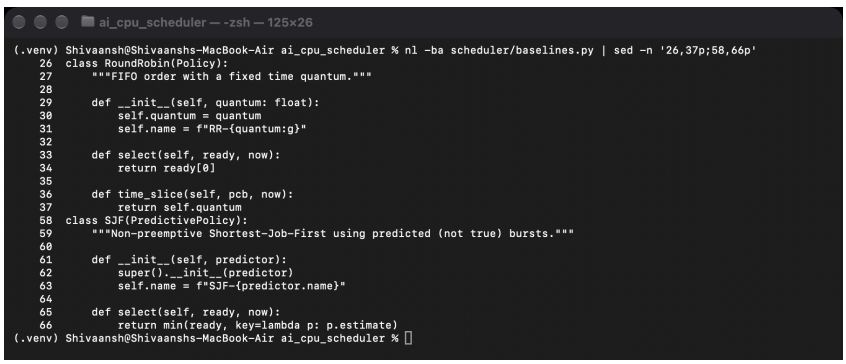


Fig. 4. Round Robin and SJF in scheduler/baselines.py

6. The prediction model

6.1 Features

For every process, 12 features are built only from bursts and I/O waits that have already finished, all of it information the operating system already has: the last 5 burst lengths, the mean and standard deviation of past bursts, the exponential average estimate, the last and mean I/O wait, the number of bursts completed, and the change between the last two bursts. A test checks that changing a process's future bursts does not change its features, which proves there is no leakage.

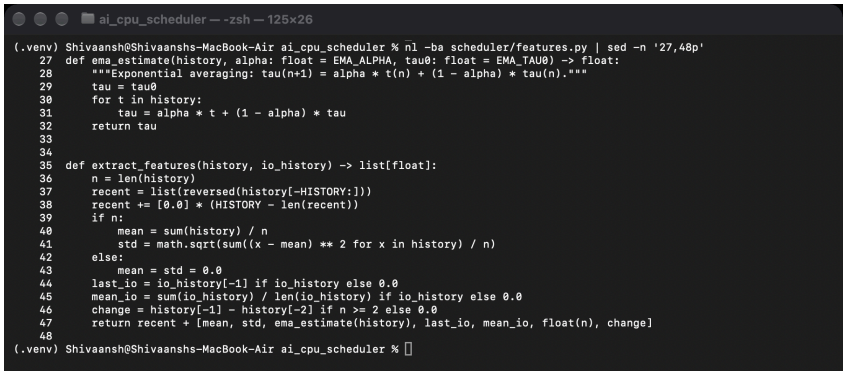


Fig. 5. Exponential averaging and the 12 features in scheduler/features.py

6.2 Training

`train_models.py` generates 2,000 processes (500 of each type) from seeds 1000–1004, which are never used in the experiments. Because the features depend only on a process's own history, the dataset (45,389 rows, one per burst) is built directly without running the simulator. It is split 75/25 **by process**, so no process appears in both training and testing.

Model	MAE (ms)	Latency per prediction	Notes
Exponential averaging	12.36	~0	The baseline to beat
Ridge regression	14.82	0.08 ms	A single linear rule cannot fit all process types
Gradient boosting	12.59	0.50 ms	Large bursts dominate the squared error
Gradient boosting, log target	**10.62**	0.50 ms	**Chosen**: 14% lower error, better for every type

Selection rule: lowest error within a 1 ms latency budget, with Ridge preferred if within 5%. Training on log(burst) helps because burst lengths range from 0.05 ms to hundreds of ms. A logistic regression classifier (macro-F1 0.94) is also saved; a random forest scored 0.94 but was 20 times slower.

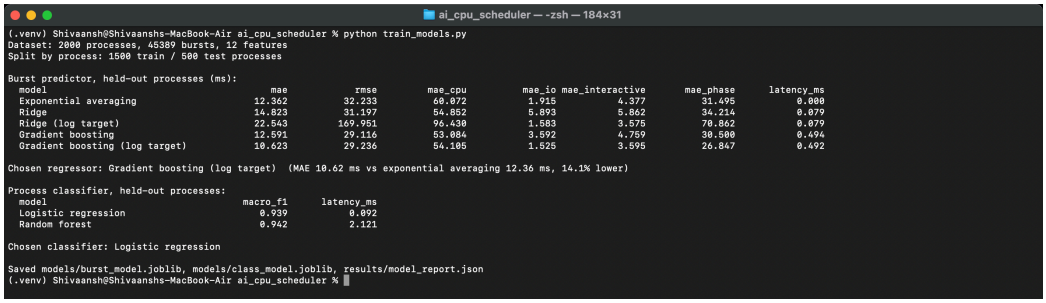


Fig. 6. Output of python train_models.py: model comparison and selection

> **Why single-threaded?** scikit-learn normally uses all CPU cores, but the simulator asks for one prediction at a time. Starting threads for one row made each prediction take 3.7 ms instead of 0.5 ms, so `scheduler/__init__.py` sets `OMP_NUM_THREADS=1`.

6.3 Feedback loop

When a burst finishes, `observe()` compares it with the prediction, logs the error (this is how prediction MAE is reported for every run) and adds the example to a rolling buffer of 4,000. Every 2,000 new examples the model is refitted on 10,000 offline rows plus the buffer. The quantum controller also keeps the last 200 ratios *actual / predicted*.

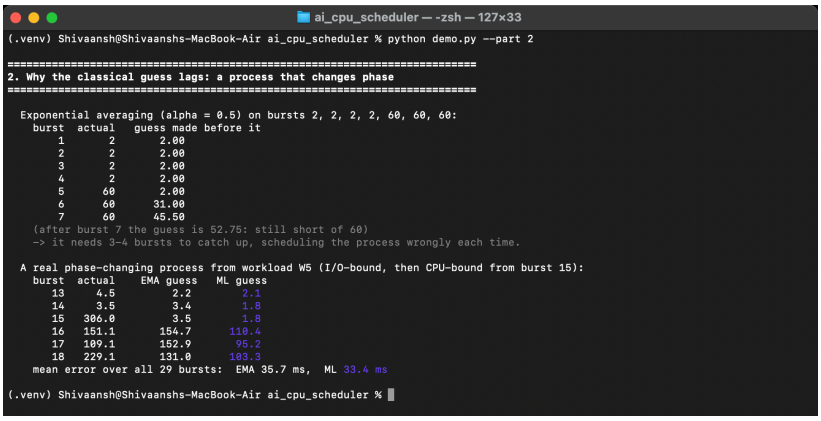


Fig. 7. Output of python demo.py --part 2: exponential averaging lagging, and EMA vs the learned model on a real process

7. The adaptive scheduler in detail

7.1 Who runs next

$$P = (\text{predicted time left in the burst}) - \beta \times (\text{time waited in the ready queue})$$

The lowest P runs. *Predicted time left* is the prediction minus the CPU time already received in this burst; if the burst has already run longer than predicted, the original prediction is used again. **Example:** with β = 0.1, process A predicted at 50 ms that has waited 400 ms has P = 50 − 40 = 10; process B predicted at 5 ms that has waited 10 ms has P = 5 − 1 = 4, so B runs. If A

had waited 600 ms, P = −10 and A would run first. With β = 0 the scheduler is exactly SJF (checked by a test). The final value β = 0.03 was picked by `tune_beta.py`.

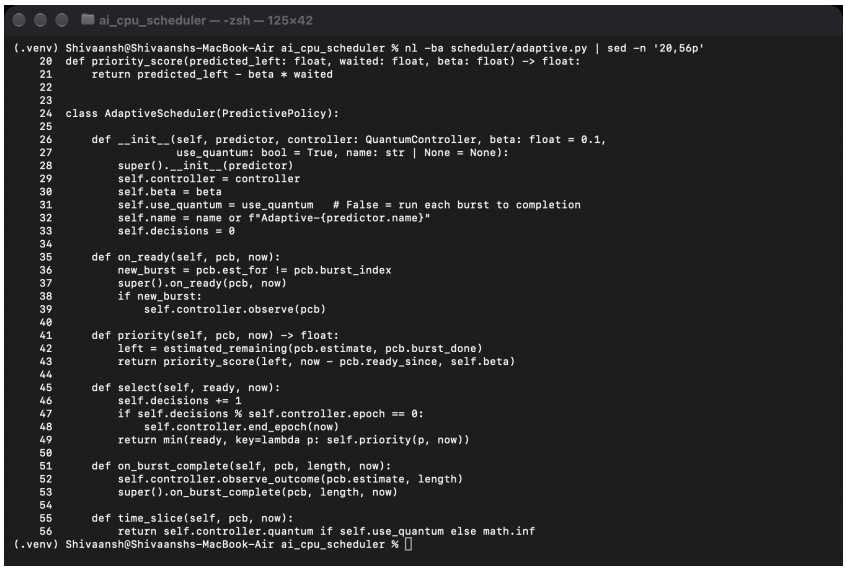


Fig. 8. The adaptive scheduler in scheduler/adaptive.py: priority with ageing, epoch trigger and quantum

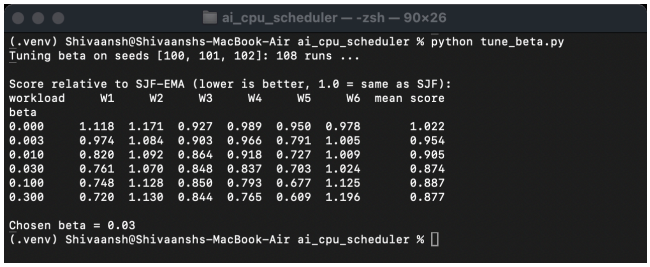


Fig. 9. Output of python tune_beta.py: choosing β on separate seeds

7.2 How long it runs

Every 50 decisions, the quantum controller takes the predictions of the bursts that became ready in that period, multiplies each by every recent error ratio, takes the 80th percentile, clamps it to 1–100 ms, and smooths it: $q = 0.3 \times q_{new} + 0.7 \times q$.

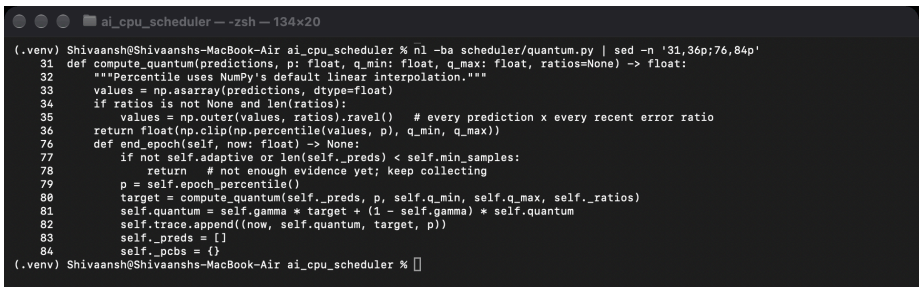


Fig. 10. The percentile rule and epoch update in scheduler/quantum.py

Worked example (I/O-bound workload). Bursts average 2 ms, so almost every prediction is about 2 ms. If the quantum used the raw predictions, the 80th percentile would be about 2 ms, too small, because 20% of real exponential bursts with mean 2 ms are longer than 3.2 ms. The recent ratios actual/predicted are spread widely, and for exponential bursts their 80th percentile is about 1.6, so the 80th percentile of 2 × ratios is about **3.2 ms**, matching the textbook rule. Starting from q = 10 ms, smoothing gives 0.3 × 3.2 + 0.7 × 10 = 7.96 ms, then 6.5, 5.5, and so on, settling near 3.5 ms. This error-ratio fix was added after the first version produced quanta that were too small on W2 and W3.

Class modulation. The classifier labels the processes seen in the epoch; the percentile becomes 80 + 10 × (fraction CPU-bound − fraction interactive), limited to 50–95. Interactive mixes get a smaller quantum (faster response), CPU-bound mixes a larger one (fewer switches).

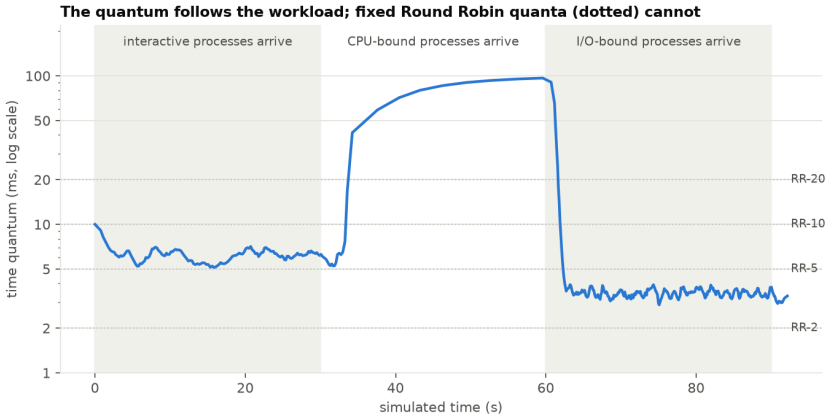


Fig. 11. The quantum on a workload that changes from interactive (about 6 ms) to CPU-bound (close to the 100 ms cap) to I/O-bound (about 3.5 ms)

8. Experiments and evaluation pipeline

Workloads. Four process types: CPU-bound (lognormal bursts, median 100 ms, 3–8 bursts), I/O-bound (exponential, mean 2 ms, 20–60 bursts), interactive (exponential, mean 5 ms, 10–40 bursts) and phase-changing (I/O-bound then CPU-bound). Six profiles of 200 processes at 85% CPU load: W1 CPU, W2 I/O, W3 interactive, W4 mixed (40/30/20/10%), W5 phase-changing, W6 = W4 at 3× arrival rate.

Configurations (11). FCFS, SJF-EMA, SRTF-EMA, RR-2, RR-5, RR-10, RR-20, MLFQ, Oracle, Adaptive-EMA (our scheduler with the classical predictor) and Adaptive-ML (our scheduler).

Runs. 6 workloads × 11 configurations × 10 seeds = **660 paired runs**, plus an ablation (120 runs), sweeps of β, p and E (130 runs) and switch costs of 0.05/0.1/0.5 ms (150 runs). The runs are spread over 9 worker processes and take about 4.5 minutes.

Pipeline.

```
train_models.py  -> models/*.joblib, results/model_report.json
tune_beta.py     -> results/beta_tuning.csv           (chooses beta =
0.03)
run_experiments.py -> results/main_results.csv, ablation.csv, sweeps.csv,
                      switch_cost.csv, quantum_trace.csv,
prediction_trace.csv
analyse.py       -> results/REPORT.md, summary_means.csv,
figures/fig1..fig8
```

Fairness of the comparison. Every policy sees the same seeds (paired comparison), the oracle gives a reference point, and expected outcomes (H1–H7) were written down before any experiment. An improvement only counts if the mean paired improvement over seeds is bigger than one standard deviation.

9. Results and what they mean

Workload	Avg wait vs SJF-EMA	Worst wait vs SJF-EMA	Seeds better	Mean quantum
W1 CPU-bound	−1.9%	−56%	7/10	92 ms
W2 I/O-bound	+13.8%	+0.5%	0/10	3.5 ms
W3 Interactive	+5.3%	−27%	2/10	6.4 ms
W4 Mixed	**−27.4%**	**−23%**	**10/10**	18 ms
W5 Phase-changing	−1.7%	−53%	4/10	95 ms
W6 Overload	+2.6%	−2%	2/10	14 ms

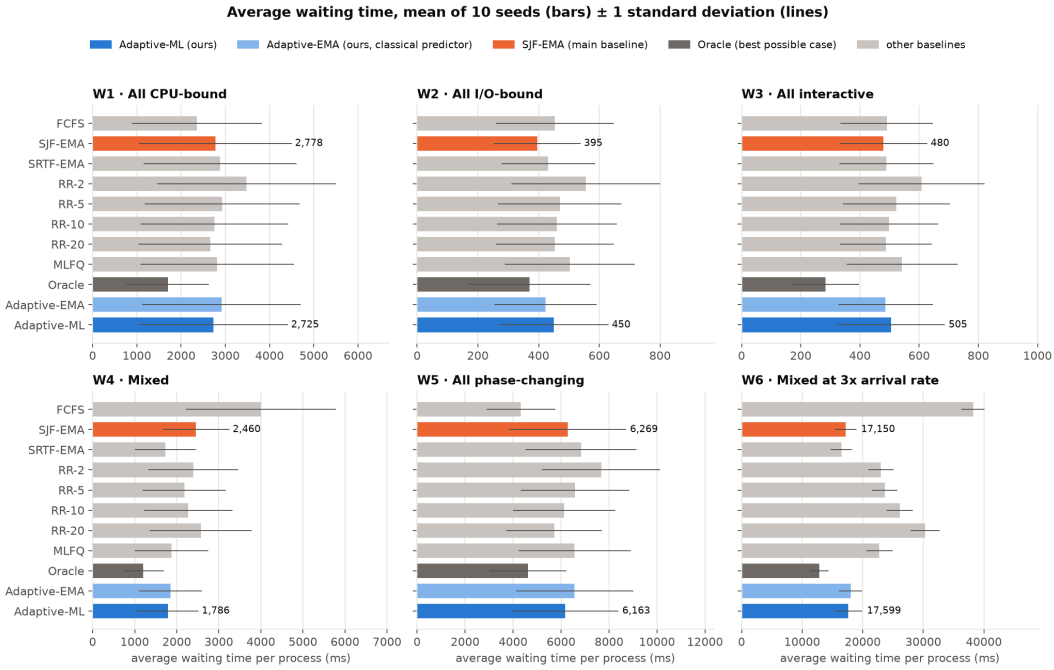


Fig. 12. Average waiting time for all 11 schedulers on the six workloads (10 seeds)

- **Where it helps:** mixed workloads, where short and long bursts compete. On W4 it beats every classical algorithm on average wait except SRTF-EMA (3% better on average, but 35% higher worst case and more switches).
- **Ageing** is what controls the worst case: removing it doubles the worst wait on W5, and starved processes on W5 drop from 7.7 (SJF-EMA) to 0.2.
- **The adaptive quantum** matters most for long bursts: fixing it at 10 ms on W5 costs 11% on average and 52% on worst case.
- **Prediction:** 12–22% lower error than EMA on every workload, but better prediction alone (ML-predicted SJF) gains only 5% on W4. The benefit comes from using the prediction both for ordering and for the quantum.
- **Where it loses:** all-I/O and all-interactive workloads have short random bursts with nothing to predict beyond the mean, so preempting at a small quantum only adds switches. Under overload, the waits are so long that ageing pushes the order towards FCFS.
- **Surprise:** on W5, plain FCFS beats even the oracle on average wait. With many bursts per process, always running the shortest burst first starves processes in their long phase. The oracle is a strong reference, not an absolute lower bound.

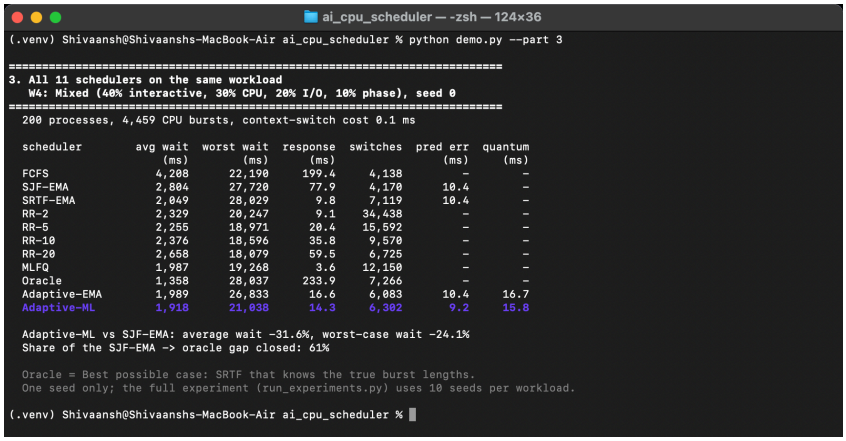


Fig. 13. Output of python demo.py --part 3: all 11 schedulers on the mixed workload (one seed)

10. How to run everything

One-time setup (Python 3.10 or newer; on this laptop it is already done):

```
cd ~/Desktop/ai_cpu_scheduler
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Every time you open a new terminal: `cd ~/Desktop/ai_cpu_scheduler && source .venv/bin/activate`

Command	Time	What it does
`pytest` (or `pytest -v`)	1 s	Runs the 37 tests
`python demo.py`	10 s	Full live demo; add `--part 1`, `2` or `3` for one part, `--workload W5` for another workload
`python train_models.py`	10 s	Retrains and compares the models
`python tune_beta.py`	70 s	Re-tunes β
`python run_experiments.py`	4.5 min	Full experiment; `--quick` for 3 seeds in under a minute
`python analyse.py`	10 s	Rebuilds REPORT.md and the figures
`sh docs/reviews/build.sh`	5 s	Rebuilds the three review PDFs (needs Google Chrome)

The demo, part by part. Part 1 shows Gantt charts for four textbook examples and checks the averages (17, 3, 5.67, 6.5 ms). Part 2 shows exponential averaging lagging (2, 31, 45.5, 52.75) and compares EMA with the learned model on a real phase-changing process. Part 3 runs all 11 schedulers on one workload and prints average wait, worst wait, response, switches, prediction error and mean quantum, with Adaptive-ML highlighted.

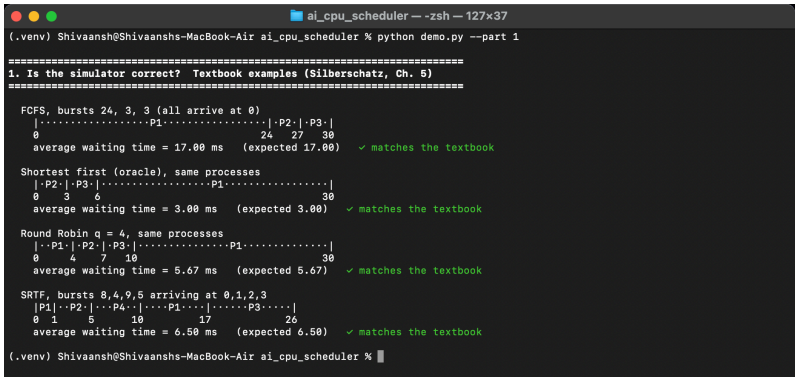


Fig. 14. Output of python demo.py --part 1: textbook checks with Gantt charts

Troubleshooting. "burst_model.joblib not found" means the models are missing: run `python train_models.py`. "command not found: python" or missing modules means the virtual environment is not active: run `source .venv/bin/activate`. Results are deterministic, so re-running gives the same numbers (only timing measurements vary).

11. Where everything is saved

Location	Contents
scheduler/	The simulator, schedulers, predictor and adaptive scheduler
tests/	The 37 tests
models/	burst_model.joblib (predictor), class_model.joblib (classifier)
results/	REPORT.md (every result and verdict), main_results.csv (660 rows, one per run), ablation, sweeps, switch-cost and trace CSVs, model_report.json, run_info.json
figures/	fig1 average wait, fig2 switches vs response, fig3 worst case, fig4 quantum over time, fig5 prediction example, fig6 ablation, fig7 sweeps, fig8 prediction error
docs/reviews/	Review_1.pdf, Review_2.pdf, Review_3.pdf and their HTML sources
docs/terminal/	Real Terminal screenshots used in the reviews
docs/guide/	This guide and the presentation transcript
docs/ (older)	project_review.pdf, ieee_paper.pdf, rendered screenshots
README.md	Short version of this guide

12. Limitations and future work

- Simulation only: one CPU, no caches or memory effects, no real kernel.
- Synthetic workloads: the model is trained and tested on the same generator (different seeds), which is also why online refit showed no benefit.
- A prediction takes about 0.5 ms in Python, far too slow for a kernel; a small decision tree or lookup table would be needed.

- Linear ageing struggles under heavy overload; scaling it with the load is a natural fix.
- Next steps: real burst traces, moving the quantum controller into xv6, and multi-core scheduling.

13. Questions to be ready for

Why use machine learning at all?

Exponential averaging uses one feature and lags after phase changes. The model uses 12 features and is 12–22% more accurate. But the honest finding is that most of the gain comes from using the prediction twice, not from accuracy alone (Adaptive-EMA vs Adaptive-ML shows this).

How do you know the simulator is correct?

It reproduces textbook results exactly (17, 3, 5.67, 6.5 ms), every scheduler delivers the same total CPU time, event times never go backwards, and 37 tests pass.

What if the prediction is wrong?

A wrong prediction only affects ordering and quantum; the process still runs. Ageing guarantees a long-waiting process eventually goes first, and the quantum uses observed error ratios, so it adjusts for systematic errors.

Why not just use SRTF?

SRTF-EMA is 3% better on average on W4 but has a 35% higher worst-case wait and more switches, because it preempts whenever a newly ready process has a shorter predicted remaining time, and it has no ageing.

How did you avoid tuning to the test data?

The model was trained on seeds 1000–1004, β was tuned on seeds 100–102, and the evaluation used seeds 0–9. All the other parameters came from the original plan.

Why does it lose on I/O-bound workloads?

Bursts are short and random (exponential), so nothing can be predicted beyond the mean, and preempting at a small quantum adds context switches without improving the order.